

CELL 1 — Install & Imports

```
!pip install kagglehub -q

import os
import random
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import seaborn as sns

import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import Adam

from sklearn.metrics import classification_report, confusion_matrix

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices('GPU'))

TensorFlow version: 2.20.0
GPU available: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]
```

CELL 2 — Download PlantVillage Dataset

```
import kagglehub

# Downloads to ~/.cache/kagglehub/datasets/...
path = kagglehub.dataset_download("abdallahalidev/plantvillage-
dataset")
print("Dataset downloaded to:", path)

# Find the folder that contains class subfolders
# PlantVillage has a nested structure; find the 'color' subfolder
BASE_DIR = None
for root, dirs, files in os.walk(path):
    if len(dirs) > 10 and not files: # class-level folder has many
subdirs
        BASE_DIR = root
        break
```

```

if BASE_DIR is None:
    BASE_DIR = path    # fallback

print("Using base directory:", BASE_DIR)
classes = sorted(os.listdir(BASE_DIR))
NUM_CLASSES = len(classes)
print(f"Found {NUM_CLASSES} classes:")
for c in classes:
    print(" ", c)

```

Using Colab cache for faster access to the 'plantvillage-dataset' dataset.

Dataset downloaded to: /kaggle/input/plantvillage-dataset

Using base directory: /kaggle/input/plantvillage-dataset/plantvillage-dataset/segmented

Found 38 classes:

```

Apple__Apple_scab
Apple__Black_rot
Apple__Cedar_apple_rust
Apple__healthy
Blueberry__healthy
Cherry_(including_sour)__Powdery_mildew
Cherry_(including_sour)__healthy
Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot
Corn_(maize)__Common_rust
Corn_(maize)__Northern_Leaf_Blight
Corn_(maize)__healthy
Grape__Black_rot
Grape__Esca_(Black_Measles)
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)
Grape__healthy
Orange__Haunglongbing_(Citrus_greening)
Peach__Bacterial_spot
Peach__healthy
Pepper,_bell__Bacterial_spot
Pepper,_bell__healthy
Potato__Early_blight
Potato__Late_blight
Potato__healthy
Raspberry__healthy
Soybean__healthy
Squash__Powdery_mildew
Strawberry__Leaf_scorch
Strawberry__healthy
Tomato__Bacterial_spot
Tomato__Early_blight
Tomato__Late_blight
Tomato__Leaf_Mold
Tomato__Septoria_leaf_spot
Tomato__Spider_mites Two-spotted_spider_mite

```

```
Tomato__Target_Spot
Tomato__Tomato_Yellow_Leaf_Curl_Virus
Tomato__Tomato_mosaic_virus
Tomato__healthy
```

CELL 3 — Data Augmentation & Preprocessing

```
IMG_SIZE    = (224, 224)
BATCH_SIZE  = 32

# --- Split into train / val / test using flow_from_directory ---
# ImageDataGenerator with augmentation for training
train_datagen = ImageDataGenerator(
    rescale          = 1.0 / 255,
    validation_split  = 0.30,           # 70% train, 30% held out
    rotation_range    = 20,
    width_shift_range = 0.15,
    height_shift_range = 0.15,
    horizontal_flip    = True,
    vertical_flip      = True,
    zoom_range         = 0.15,
    brightness_range   = [0.8, 1.2],
    fill_mode          = 'nearest',
)

# No augmentation for val/test – only rescale
val_test_datagen = ImageDataGenerator(
    rescale          = 1.0 / 255,
    validation_split  = 0.30,
)

train_gen = train_datagen.flow_from_directory(
    BASE_DIR,
    target_size      = IMG_SIZE,
    batch_size       = BATCH_SIZE,
    class_mode       = 'categorical',
    subset           = 'training',
    seed             = SEED,
    shuffle           = True,
)

# Use the held-out 30% and split it into val (20%) and test (10%)
# We'll do this by generating two generators with different subsets
# and taking a slice for test later
val_gen = val_test_datagen.flow_from_directory(
    BASE_DIR,
    target_size      = IMG_SIZE,
    batch_size       = BATCH_SIZE,
    class_mode       = 'categorical',
```

```

subset      = 'validation',
seed        = SEED,
shuffle     = False,
)

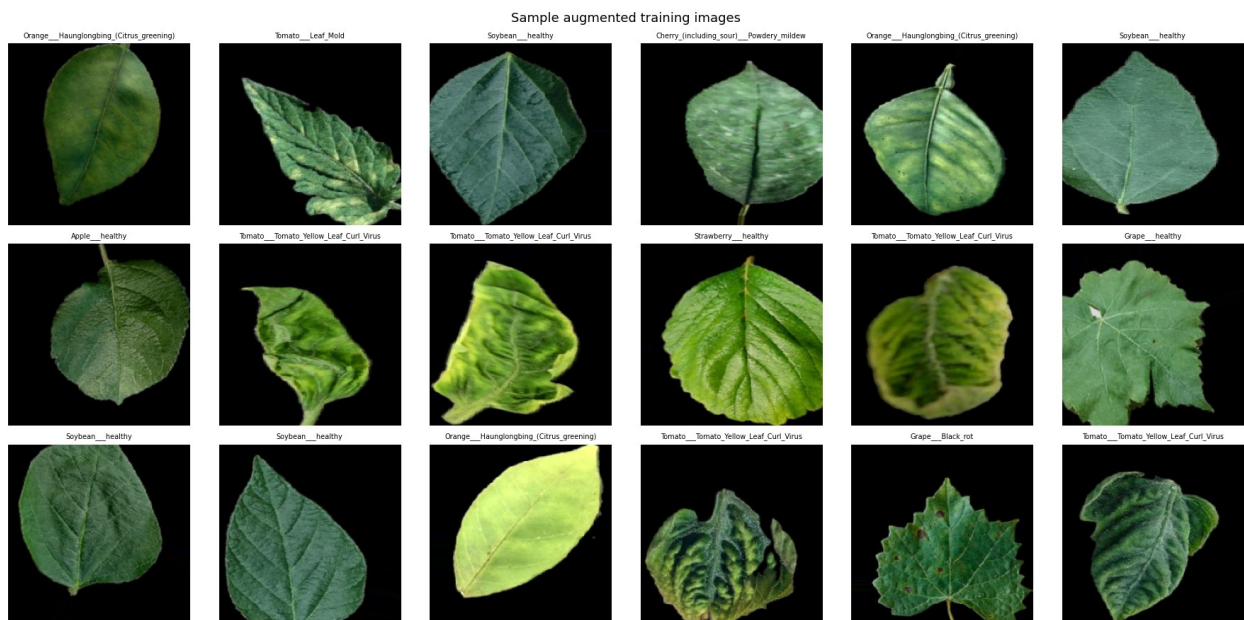
print(f"\nTraining samples : {train_gen.samples}")
print(f"Validation samples : {val_gen.samples}")

# Visualise a batch of augmented images
sample_images, sample_labels = next(train_gen)
fig, axes = plt.subplots(3, 6, figsize=(18, 9))
for i, ax in enumerate(axes.flat):
    ax.imshow(sample_images[i])
    ax.set_title(classes[np.argmax(sample_labels[i])], fontsize=7)
    ax.axis('off')
plt.suptitle("Sample augmented training images", fontsize=13)
plt.tight_layout()
plt.show()

```

Found 38029 images belonging to 38 classes.
Found 16277 images belonging to 38 classes.

Training samples : 38029
Validation samples : 16277



CELL 4 — Build CNN Architecture from Scratch

```

def conv_block(x, filters, pool=True):
    """Conv → BN → ReLU (optionally followed by MaxPool)."""

```

```

x = layers.Conv2D(filters, 3, padding='same', use_bias=False)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = layers.Conv2D(filters, 3, padding='same', use_bias=False)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
if pool:
    x = layers.MaxPooling2D(2)(x)
return x

def build_cnn(input_shape=(224, 224, 3), num_classes=38):
    inputs = layers.Input(shape=input_shape)

    # Block 1 – 32 filters
    x = conv_block(inputs, 32)          # → 112×112×32

    # Block 2 – 64 filters
    x = conv_block(x, 64)              # → 56×56×64

    # Block 3 – 128 filters
    x = conv_block(x, 128)            # → 28×28×128

    # Block 4 – 256 filters
    x = conv_block(x, 256)            # → 14×14×256

    # Block 5 – 512 filters (no pool – use GlobalAvgPool next)
    x = conv_block(x, 512, pool=False) # → 14×14×512

    # Global Average Pooling replaces Flatten → fewer params, less
    overfit
    x = layers.GlobalAveragePooling2D()(x) # → 512

    # Classifier head
    x = layers.Dense(256, use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Dropout(0.5)(x)

    outputs = layers.Dense(num_classes, activation='softmax')(x)

    return models.Model(inputs, outputs, name='PlantVillage_CNN')

model = build_cnn(num_classes=NUM_CLASSES)
model.summary()

# Count trainable parameters
total_params = model.count_params()
print(f"\nTotal parameters: {total_params:,}")

Model: "PlantVillage_CNN"

```

Layer (type) Param #	Output Shape	
input_layer (InputLayer) 0	(None, 224, 224, 3)	
conv2d (Conv2D) 864	(None, 224, 224, 32)	
batch_normalization 128 (BatchNormalization)	(None, 224, 224, 32)	
activation (Activation) 0	(None, 224, 224, 32)	
conv2d_1 (Conv2D) 9,216	(None, 224, 224, 32)	
batch_normalization_1 128 (BatchNormalization)	(None, 224, 224, 32)	
activation_1 (Activation) 0	(None, 224, 224, 32)	
max_pooling2d (MaxPooling2D) 0	(None, 112, 112, 32)	
conv2d_2 (Conv2D) 18,432	(None, 112, 112, 64)	
batch_normalization_2 256 (BatchNormalization)	(None, 112, 112, 64)	

0	activation_2 (Activation)	(None, 112, 112, 64)
36,864	conv2d_3 (Conv2D)	(None, 112, 112, 64)
256	batch_normalization_3 (BatchNormalization)	(None, 112, 112, 64)
0	activation_3 (Activation)	(None, 112, 112, 64)
0	max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)
73,728	conv2d_4 (Conv2D)	(None, 56, 56, 128)
512	batch_normalization_4 (BatchNormalization)	(None, 56, 56, 128)
0	activation_4 (Activation)	(None, 56, 56, 128)
147,456	conv2d_5 (Conv2D)	(None, 56, 56, 128)
512	batch_normalization_5 (BatchNormalization)	(None, 56, 56, 128)
0	activation_5 (Activation)	(None, 56, 56, 128)

0	max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)
294,912	conv2d_6 (Conv2D)	(None, 28, 28, 256)
1,024	batch_normalization_6 (BatchNormalization)	(None, 28, 28, 256)
0	activation_6 (Activation)	(None, 28, 28, 256)
589,824	conv2d_7 (Conv2D)	(None, 28, 28, 256)
1,024	batch_normalization_7 (BatchNormalization)	(None, 28, 28, 256)
0	activation_7 (Activation)	(None, 28, 28, 256)
0	max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 256)
1,179,648	conv2d_8 (Conv2D)	(None, 14, 14, 512)
2,048	batch_normalization_8 (BatchNormalization)	(None, 14, 14, 512)
0	activation_8 (Activation)	(None, 14, 14, 512)

conv2d_9 (Conv2D)	(None, 14, 14, 512)	
2,359,296		
batch_normalization_9	(None, 14, 14, 512)	
2,048		
(BatchNormalization)		
activation_9 (Activation)	(None, 14, 14, 512)	
0		
global_average_pooling2d	(None, 512)	
0		
(GlobalAveragePooling2D)		
dense (Dense)	(None, 256)	
131,072		
batch_normalization_10	(None, 256)	
1,024		
(BatchNormalization)		
activation_10 (Activation)	(None, 256)	
0		
dropout (Dropout)	(None, 256)	
0		
dense_1 (Dense)	(None, 38)	
9,766		

Total params: 4,860,038 (18.54 MB)

Trainable params: 4,855,558 (18.52 MB)

Non-trainable params: 4,480 (17.50 KB)

Total parameters: 4,860,038

CELL 5 — Compile Model

```
LEARNING_RATE = 1e-3

model.compile(
    optimizer = Adam(learning_rate=LEARNING_RATE),
    loss      = 'categorical_crossentropy',
    metrics   = ['accuracy'],
)

print("Model compiled successfully.")
print(f"  Optimizer : Adam (lr={LEARNING_RATE})")
print(f"  Loss      : categorical_crossentropy")
print(f"  Metrics   : accuracy")

Model compiled successfully.
Optimizer : Adam (lr=0.001)
Loss      : categorical_crossentropy
Metrics   : accuracy
```

CELL 6 — Train with Callbacks

```
EPOCHS      = 20
CKPT_PATH    = 'best_plantvillage_cnn.keras'

cb_list = [
    # Stop if val_accuracy doesn't improve for 8 epochs
    callbacks.EarlyStopping(
        monitor      = 'val_accuracy',
        patience     = 8,
        restore_best_weights = True,
        verbose      = 1,
    ),
    # Halve LR when val_loss plateaus for 4 epochs
    callbacks.ReduceLROnPlateau(
        monitor      = 'val_loss',
        factor        = 0.3,
        patience     = 4,
        min_lr       = 1e-7,
        verbose      = 1,
    ),
    # Save best weights
    callbacks.ModelCheckpoint(
        filepath      = CKPT_PATH,
        monitor       = 'val_accuracy',
        save_best_only = True,
    ),
]
```

```

        save_weights_only = False,
        verbose            = 1,
    ),
]

history = model.fit(
    train_gen,
    epochs      = EPOCHS,
    validation_data = val_gen,
    callbacks    = cb_list,
    verbose      = 1,
)

print("\nTraining complete.")
print(f"Best val accuracy :
{max(history.history['val_accuracy']):.4f}")

Epoch 1/20
1189/1189 _____ 0s 614ms/step - accuracy: 0.4031 -
loss: 2.2028
Epoch 1: val_accuracy improved from None to 0.24224, saving model to
best_plantvillage_cnn.keras

Epoch 1: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 848s 689ms/step - accuracy: 0.5237 -
loss: 1.6699 - val_accuracy: 0.2422 - val_loss: 3.4990 -
learning_rate: 0.0010
Epoch 2/20
1189/1189 _____ 0s 513ms/step - accuracy: 0.7079 -
loss: 0.9543
Epoch 2: val_accuracy improved from 0.24224 to 0.61117, saving model
to best_plantvillage_cnn.keras

Epoch 2: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 644s 542ms/step - accuracy: 0.7374 -
loss: 0.8531 - val_accuracy: 0.6112 - val_loss: 1.3052 -
learning_rate: 0.0010
Epoch 3/20
1189/1189 _____ 0s 508ms/step - accuracy: 0.7976 -
loss: 0.6410
Epoch 3: val_accuracy improved from 0.61117 to 0.69141, saving model
to best_plantvillage_cnn.keras

Epoch 3: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 634s 534ms/step - accuracy: 0.8066 -
loss: 0.6112 - val_accuracy: 0.6914 - val_loss: 1.0631 -
learning_rate: 0.0010
Epoch 4/20
1189/1189 _____ 0s 510ms/step - accuracy: 0.8338 -
loss: 0.5218

```

Epoch 4: val_accuracy did not improve from 0.69141
1189/1189 _____ 636s 535ms/step - accuracy: 0.8435 -
loss: 0.4920 - val_accuracy: 0.5879 - val_loss: 1.8952 -
learning_rate: 0.0010

Epoch 5/20
1189/1189 _____ 0s 512ms/step - accuracy: 0.8651 -
loss: 0.4250

Epoch 5: val_accuracy improved from 0.69141 to 0.85470, saving model
to best_plantvillage_cnn.keras

Epoch 5: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 640s 538ms/step - accuracy: 0.8708 -
loss: 0.4040 - val_accuracy: 0.8547 - val_loss: 0.4479 -
learning_rate: 0.0010

Epoch 6/20
1189/1189 _____ 0s 506ms/step - accuracy: 0.8858 -
loss: 0.3491

Epoch 6: val_accuracy did not improve from 0.85470
1189/1189 _____ 635s 534ms/step - accuracy: 0.8885 -
loss: 0.3439 - val_accuracy: 0.7275 - val_loss: 1.0766 -
learning_rate: 0.0010

Epoch 7/20
1189/1189 _____ 0s 497ms/step - accuracy: 0.9011 -
loss: 0.3113

Epoch 7: val_accuracy improved from 0.85470 to 0.89998, saving model
to best_plantvillage_cnn.keras

Epoch 7: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 621s 522ms/step - accuracy: 0.9024 -
loss: 0.3040 - val_accuracy: 0.9000 - val_loss: 0.2942 -
learning_rate: 0.0010

Epoch 8/20
1189/1189 _____ 0s 503ms/step - accuracy: 0.9157 -
loss: 0.2655

Epoch 8: val_accuracy did not improve from 0.89998
1189/1189 _____ 628s 528ms/step - accuracy: 0.9135 -
loss: 0.2671 - val_accuracy: 0.7808 - val_loss: 0.7275 -
learning_rate: 0.0010

Epoch 9/20
1189/1189 _____ 0s 511ms/step - accuracy: 0.9220 -
loss: 0.2426

Epoch 9: val_accuracy improved from 0.89998 to 0.93267, saving model
to best_plantvillage_cnn.keras

Epoch 9: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 638s 537ms/step - accuracy: 0.9227 -
loss: 0.2416 - val_accuracy: 0.9327 - val_loss: 0.2014 -
learning_rate: 0.0010

Epoch 10/20
1189/1189 _____ 0s 505ms/step - accuracy: 0.9289 -

```
loss: 0.2196
Epoch 10: val_accuracy did not improve from 0.93267
1189/1189 _____ 634s 533ms/step - accuracy: 0.9283 -
loss: 0.2217 - val_accuracy: 0.8377 - val_loss: 0.5848 -
learning_rate: 0.0010
Epoch 11/20
1189/1189 _____ 0s 504ms/step - accuracy: 0.9324 -
loss: 0.2134
Epoch 11: val_accuracy did not improve from 0.93267
1189/1189 _____ 631s 530ms/step - accuracy: 0.9342 -
loss: 0.2053 - val_accuracy: 0.9027 - val_loss: 0.3143 -
learning_rate: 0.0010
Epoch 12/20
1189/1189 _____ 0s 504ms/step - accuracy: 0.9396 -
loss: 0.1842
Epoch 12: val_accuracy did not improve from 0.93267
1189/1189 _____ 630s 530ms/step - accuracy: 0.9391 -
loss: 0.1874 - val_accuracy: 0.7468 - val_loss: 1.1568 -
learning_rate: 0.0010
Epoch 13/20
1189/1189 _____ 0s 501ms/step - accuracy: 0.9452 -
loss: 0.1691
Epoch 13: ReduceLROnPlateau reducing learning rate to
0.0003000000142492354.

Epoch 13: val_accuracy did not improve from 0.93267
1189/1189 _____ 625s 526ms/step - accuracy: 0.9450 -
loss: 0.1678 - val_accuracy: 0.8285 - val_loss: 0.6427 -
learning_rate: 0.0010
Epoch 14/20
1189/1189 _____ 0s 507ms/step - accuracy: 0.9626 -
loss: 0.1161
Epoch 14: val_accuracy improved from 0.93267 to 0.96934, saving model
to best_plantvillage_cnn.keras

Epoch 14: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 632s 532ms/step - accuracy: 0.9674 -
loss: 0.1009 - val_accuracy: 0.9693 - val_loss: 0.0938 -
learning_rate: 3.0000e-04
Epoch 15/20
1189/1189 _____ 0s 497ms/step - accuracy: 0.9690 -
loss: 0.0936
Epoch 15: val_accuracy did not improve from 0.96934
1189/1189 _____ 620s 522ms/step - accuracy: 0.9698 -
loss: 0.0923 - val_accuracy: 0.9577 - val_loss: 0.1292 -
learning_rate: 3.0000e-04
Epoch 16/20
1189/1189 _____ 0s 500ms/step - accuracy: 0.9720 -
loss: 0.0867
```

```

Epoch 16: val_accuracy did not improve from 0.96934
1189/1189 _____ 624s 525ms/step - accuracy: 0.9715 -
loss: 0.0879 - val_accuracy: 0.9532 - val_loss: 0.1469 -
learning_rate: 3.0000e-04
Epoch 17/20
1189/1189 _____ 0s 501ms/step - accuracy: 0.9738 -
loss: 0.0811
Epoch 17: val_accuracy improved from 0.96934 to 0.97567, saving model
to best_plantvillage_cnn.keras

Epoch 17: finished saving model to best_plantvillage_cnn.keras
1189/1189 _____ 625s 526ms/step - accuracy: 0.9743 -
loss: 0.0791 - val_accuracy: 0.9757 - val_loss: 0.0730 -
learning_rate: 3.0000e-04
Epoch 18/20
1189/1189 _____ 0s 503ms/step - accuracy: 0.9737 -
loss: 0.0800
Epoch 18: val_accuracy did not improve from 0.97567
1189/1189 _____ 628s 528ms/step - accuracy: 0.9738 -
loss: 0.0784 - val_accuracy: 0.9676 - val_loss: 0.1005 -
learning_rate: 3.0000e-04
Epoch 19/20
1189/1189 _____ 0s 508ms/step - accuracy: 0.9749 -
loss: 0.0757
Epoch 19: val_accuracy did not improve from 0.97567
1189/1189 _____ 636s 535ms/step - accuracy: 0.9745 -
loss: 0.0770 - val_accuracy: 0.9669 - val_loss: 0.1039 -
learning_rate: 3.0000e-04
Epoch 20/20
1189/1189 _____ 0s 503ms/step - accuracy: 0.9781 -
loss: 0.0708
Epoch 20: val_accuracy did not improve from 0.97567
1189/1189 _____ 629s 529ms/step - accuracy: 0.9768 -
loss: 0.0727 - val_accuracy: 0.9631 - val_loss: 0.1175 -
learning_rate: 3.0000e-04
Restoring model weights from the end of the best epoch: 17.

Training complete.
Best val accuracy : 0.9757

```

CELL 7 — Plot Training Curves

```

acc      = history.history['accuracy']
val_acc  = history.history['val_accuracy']
loss     = history.history['loss']
val_loss = history.history['val_loss']
epochs_run = range(1, len(acc) + 1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

```

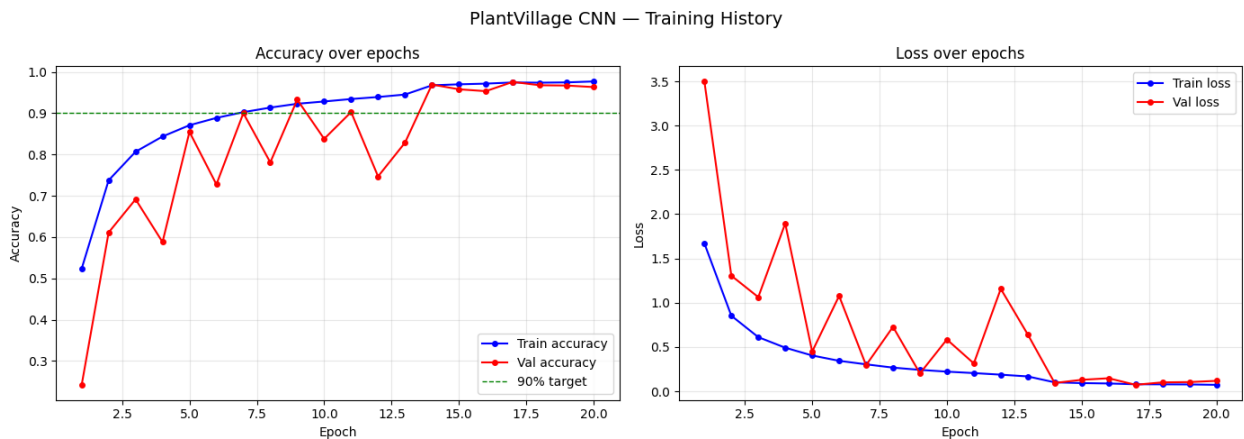
```

# Accuracy
ax1.plot(epochs_ran, acc,      'b-o', markersize=4, label='Train
accuracy')
ax1.plot(epochs_ran, val_acc, 'r-o', markersize=4, label='Val
accuracy')
ax1.axhline(y=0.90, color='green', linestyle='--', linewidth=1,
label='90% target')
ax1.set_title('Accuracy over epochs')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Loss
ax2.plot(epochs_ran, loss,      'b-o', markersize=4, label='Train
loss')
ax2.plot(epochs_ran, val_loss, 'r-o', markersize=4, label='Val loss')
ax2.set_title('Loss over epochs')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.suptitle('PlantVillage CNN – Training History', fontsize=14)
plt.tight_layout()
plt.show()

```



CELL 8 — Evaluate on Test Set & Classification Report

```

# Load the best saved model
best_model = tf.keras.models.load_model(CKPT_PATH)
print(f"Loaded best model from: {CKPT_PATH}")

# Use the val_gen as the test set (shuffle=False, so labels are

```

```

aligned)
# Reset generator to start from beginning
val_gen.reset()

# Evaluate
test_loss, test_acc = best_model.evaluate(val_gen, verbose=1)
print(f"\nTest Loss      : {test_loss:.4f}")
print(f"Test Accuracy : {test_acc:.4f}  ({test_acc*100:.2f}%)")

# Predict all test samples
val_gen.reset()
y_pred_probs = best_model.predict(val_gen, verbose=1)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = val_gen.classes

# --- Classification Report ---
print("\n" + "="*60)
print("CLASSIFICATION REPORT")
print("="*60)
print(classification_report(
    y_true, y_pred,
    target_names = classes,
    digits       = 3,
))

# --- Confusion Matrix Heatmap ---
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(20, 18))
sns.heatmap(
    cm,
    annot      = True,
    fmt        = 'd',
    cmap       = 'Blues',
    xticklabels= classes,
    yticklabels= classes,
    linewidths = 0.5,
)
plt.title('Confusion Matrix – PlantVillage CNN', fontsize=15)
plt.xlabel('Predicted label', fontsize=12)
plt.ylabel('True label', fontsize=12)
plt.xticks(rotation=90, fontsize=7)
plt.yticks(rotation=0,  fontsize=7)
plt.tight_layout()
plt.show()

# --- Per-class accuracy summary ---
per_class_acc = cm.diagonal() / cm.sum(axis=1)
print("\nPer-class accuracy (top 5 lowest – check these first):")
sorted_idx = np.argsort(per_class_acc)

```



```
for i in sorted_idx[:5]:
    print(f" {classes[i]:<45} {per_class_acc[i]*100:.1f}%")
```

Loaded best model from: best_plantvillage_cnn.keras
 509/509 _____ 32s 60ms/step - accuracy: 0.9757 - loss: 0.0730

Test Loss : 0.0730
 Test Accuracy : 0.9757 (97.57%)
 509/509 _____ 33s 62ms/step

```
=====
CLASSIFICATION REPORT
=====
```

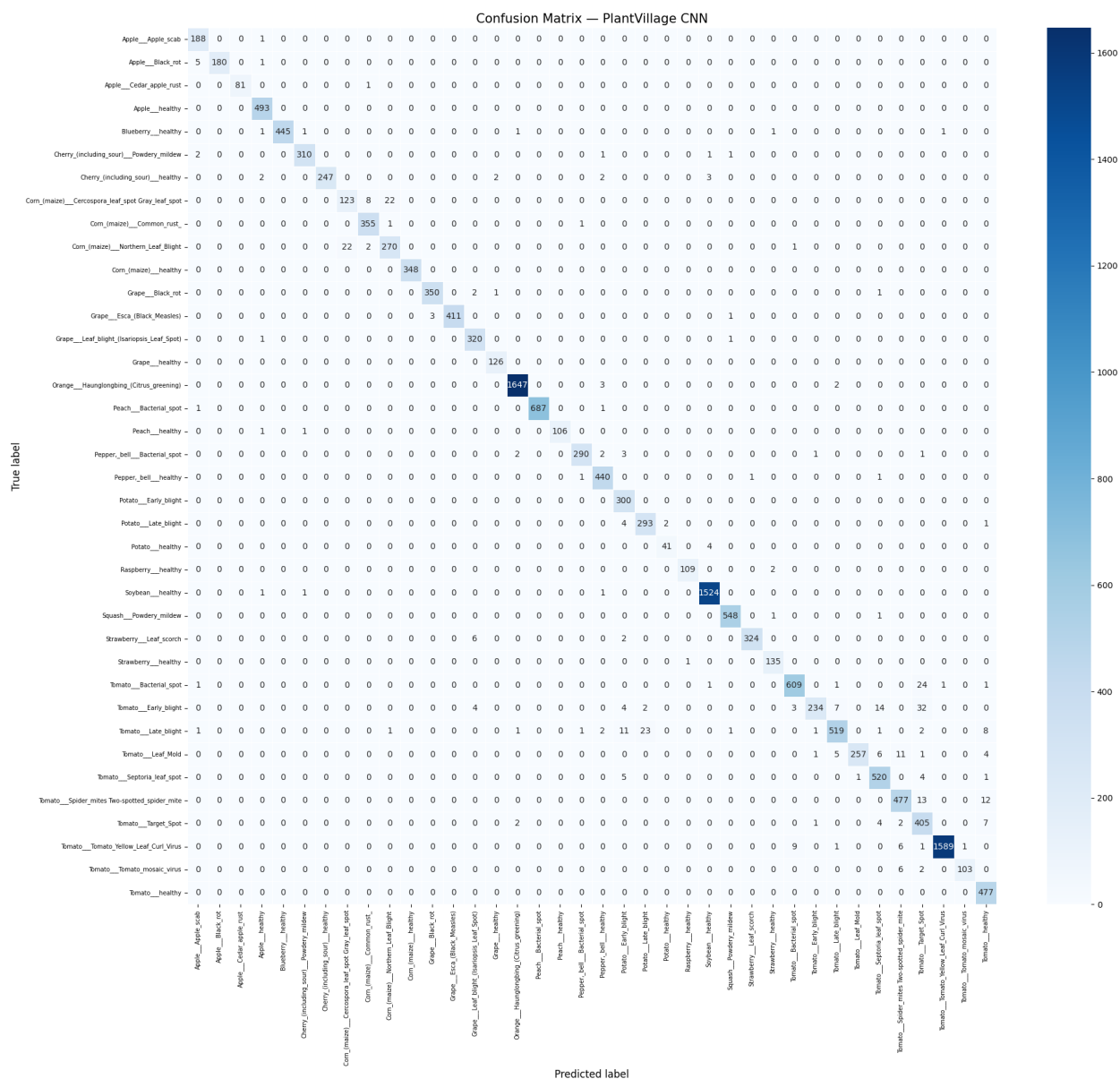
			precision
recall	f1-score	support	
		Apple___Apple_scab	0.949
0.995	0.972	189	
		Apple___Black_rot	1.000
0.968	0.984	186	
		Apple___Cedar_apple_rust	1.000
0.988	0.994	82	
		Apple___healthy	0.984
1.000	0.992	493	
		Blueberry___healthy	1.000
0.989	0.994	450	
		Cherry_(including_sour)___Powdery_mildew	0.990
0.984	0.987	315	
		Cherry_(including_sour)___healthy	1.000
0.965	0.982	256	
		Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot	0.848
0.804	0.826	153	
		Corn_(maize)___Common_rust_	0.970
0.994	0.982	357	
		Corn_(maize)___Northern_Leaf_Blight	0.918
0.915	0.917	295	
		Corn_(maize)___healthy	1.000
1.000	1.000	348	
		Grape___Black_rot	0.992
0.989	0.990	354	
		Grape___Esca_(Black_Measles)	1.000
0.990	0.995	415	
		Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	0.964
0.994	0.979	322	
		Grape___healthy	0.977
1.000	0.988	126	
		Orange___Haunglongbing_(Citrus_greening)	0.996
0.997	0.997	1652	
		Peach___Bacterial_spot	1.000

0.997	0.999	689		
			Peach___healthy	1.000
0.981	0.991	108		
			Pepper,_bell___Bacterial_spot	0.990
0.970	0.980	299		
			Pepper,_bell___healthy	0.973
0.993	0.983	443		
			Potato___Early_blight	0.912
1.000	0.954	300		
			Potato___Late_blight	0.921
0.977	0.948	300		
			Potato___healthy	0.953
0.911	0.932	45		
			Raspberry___healthy	0.991
0.982	0.986	111		
			Soybean___healthy	0.994
0.998	0.996	1527		
			Squash___Powdery_mildew	0.993
0.996	0.995	550		
			Strawberry___Leaf_scorch	0.997
0.976	0.986	332		
			Strawberry___healthy	0.971
0.993	0.982	136		
			Tomato___Bacterial_spot	0.979
0.955	0.967	638		
			Tomato___Early_blight	0.983
0.780	0.870	300		
			Tomato___Late_blight	0.970
0.907	0.938	572		
			Tomato___Leaf_Mold	0.996
0.902	0.947	285		
			Tomato___Septoria_leaf_spot	0.949
0.979	0.964	531		
			Tomato___Spider_mites Two-spotted_spider_mite	0.950
0.950	0.950	502		
			Tomato___Target_Spot	0.835
0.962	0.894	421		
			Tomato___Tomato_Yellow_Leaf_Curl_Virus	0.999
0.989	0.994	1607		
			Tomato___Tomato_mosaic_virus	0.990
0.928	0.958	111		
			Tomato___healthy	0.933
1.000	0.966	477		
			accuracy	
0.976	16277			
			macro avg	0.970
0.966	0.967	16277		
			weighted avg	0.977

0.976

0.976

16277



Per-class accuracy (top 5 lowest – check these first):

Tomato___Early_blight 78.0%
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot 80.4%
Tomato___Leaf_Mold 90.2%
Tomato___Late_blight 90.7%
Potato___healthy 91.1%